Reproducibility Guide — EMS Readiness Optimization

Why Reproducibility Matters

The EMS simulation uses stochastic processes (NHPP arrivals, random service times, dispatch tie-breaking). Without careful seed management:
- Results change between runs, making debugging impossible.
- Published results can’t be verified.
- A/B comparisons of policies are confounded by randomness.

Architecture

SeedManager (`src/ems_readiness/utils/reproducibility.py`)

Central class that provides **deterministic, isolated** random number generators.

```
Master Seed (42)
  ├── hash("arrivals")   → RNG for arrival generation
  ├── hash("service")    → RNG for service times
  ├── hash("dispatch")   → RNG for dispatch tie-breaking
  └── hash("simulation") → RNG for SimPy engine
```

Key properties:
- **Deterministic**: Same master seed → same derived seeds → same results.
- **Isolated**: Changing the arrival stream doesn’t affect service-time draws.
- **Cached**: Requesting the same component twice returns the same RNG instance.

Configuration

```yaml
# configs/reproducibility.yaml
reproducibility:
  master_seed: 42
  components:
    - arrivals
    - service
    - dispatch
    - simulation
```

Override via environment variable:

```
export EMS_MASTER_SEED=12345
python run_simulation.py
```

Usage

Basic

```
from ems_readiness.utils.reproducibility import SeedManager

sm = SeedManager(master_seed=42)
rng = sm.get_rng("arrivals")
values = rng.random(10)  # always the same 10 floats
```

From Config File

```
sm = SeedManager.from_config("configs/reproducibility.yaml")
```

Batch Replications

For N replications with different but reproducible seeds:

```
for rep in range(10):
    sm = SeedManager(master_seed=42 + rep)
    engine = SimulationEngine(seed=sm.get_seed("simulation"))
    engine.run()
```

This matches the existing `BatchRunner` pattern (`seed_base + rep`).

Recording Metadata

```
meta = sm.get_metadata()
# {'master_seed': 42, 'components': {'arrivals': 17239847, ...}}
import json
with open("results/run_metadata.json", "w") as f:
    json.dump(meta, f, indent=2)
```

Reproducing a Past Experiment

1. Find the `master_seed` from the run's metadata or config.
2. Set it in `configs/reproducibility.yaml` or `EMS_MASTER_SEED` .
3. **Set the correct capacity parameter** for the experiment regime:
   - **v1 experiments** (`results/simulation/production/`): use `capacity=5` in `configs/optimization.yaml`
   - **v2 experiments** (`results/production_v2/`): use `capacity=2` in `configs/optimization.yaml` (current default)
4. Run the same code version (check the git commit hash).
5. Results will be bit-for-bit identical.

> **Important:** The project default capacity was changed from 5 to 2 during development. To reproduce v1 results, you must temporarily set `firehouse_capacity: 5` in `configs/optimization.yaml` .

Integration with Existing Code

The simulation engine (`engine.py`) already accepts a `seed` parameter:

```
engine = SimulationEngine(
    config=config,
    allocation=allocation,
    demand_data=demand_data,
    seed=sm.get_seed("simulation")
)
```

The `BatchRunner` (`runner.py`) already uses `seed_base + rep` .
To integrate SeedManager, replace the literal seed base:

```
sm = SeedManager(master_seed=config.get("master_seed", 42))
for rep in range(n_replications):
    rep_sm = SeedManager(master_seed=sm.get_seed("simulation") + rep)
    # ...
```

Best Practices

1. **Always record the master seed** alongside results.
2. **Never share RNGs** between independent components.
3. **Pin the git commit** — code changes can alter call order even with same seed.
4. **Use the environment variable** for CI/CD sweeps over seeds.
5. **Run ≥ 30 replications** for publishable statistics; report the seed range.

Data Versioning

The data pipeline now generates a manifest file (`data/processed/.data_manifest.json`)
after each successful run, which tracks:

- **Timestamp** of generation
- **Git commit hash** at the time of generation
- **Raw data file hashes** (SHA-256) for all input files
- **Seed** used for the pipeline
- **Software versions** (Python, pandas, numpy, geopandas, etc.)
- **Platform** information (OS, architecture)

Viewing the Current Data Version

```
python scripts/generate_all_data.py --version
```

This displays a summary like:

```
EMS Data Version Info
========================================
Generated at:  2026-03-15T23:46:06+00:00
Git commit:    aac03c9b
Seed:          42

Software versions:
  python           3.11.6
  pandas           2.2.3
  numpy            1.26.4
  ...
```

Comparing Manifests

To check if data needs regeneration (e.g., after updating raw files or upgrading packages):

```python
from scripts.data_processing.versioning import DataVersionManager
dvm = DataVersionManager(".")
result = dvm.compare_manifests()
if result["needs_regeneration"]:
    for reason in result["reasons"]:
        print(f"  - {reason}")
```

Smart Caching

The pipeline uses a cache manifest (`.cache_manifest.json`) to track input file hashes per tier. On subsequent runs, if inputs have not changed, the tier is skipped entirely. This can reduce a typical re-run from minutes to under a second.

Use `--force` to bypass caching, or `--no-cache` to disable it without force-regenerating.